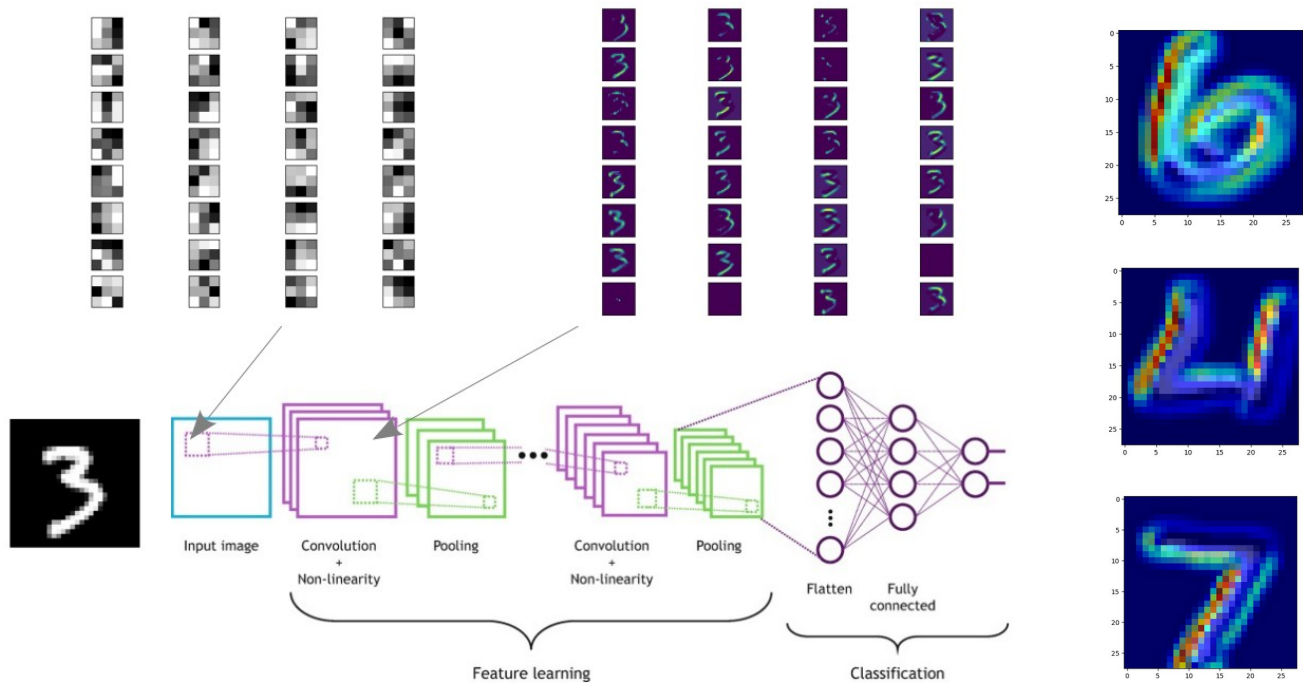


CNN filter, features and Gradcam visualisations

Objective: In this practice, students will learn to inspect the filter learned by Convolutional Neural Network trained for MNIST and FashionMNIST datasets. Furthermore, basic GradCam algorithm will be applied to highlight the method focus. Additionally, small practice to apply GradCam to pretrained model is provided. The practice will be done using Keras framework and built-up from the previous practices.



Left figure above show examples of the visualisations of the learned filter (top left) and respective features (top right) during the training to identify the class label of the input. Right figure, shows example of area of image under focus with respect to the gradient calculated (GradCam). Understanding on how to explore these learned features will provide more understanding of the inner working of the CNN method that students build and also reinforce understanding of the structure that inherently constructed during training.

Practices: Open **FilterGradCam_basic.py** and follow the comments for the exercises.

1. **Application 1** is to train a basic CNN model with the steps:

1. call the function `train()` on the main function.

```
train()
```

2. Use `process_data()` to retrieve MNIST.

```
x_train,y_train, x_test, y_test,input_shape, num_classes = process_data()
```

3. Define the CNN model.

```
model = CNN(input_shape,num_classes)
```

4. Show the summary of the model.

```
print(model.summary())
```

5. Fit the model and also save the training history.

```
history = model.fit(x_train,y_train,batch_size = 32,epochs = 10,verbose = 1,validation_data =  
(x_test, y_test))
```

6. plot the history using plot_loss function.

```
plot_loss(history)
```

7. Evaluate the model loss and accuracy.

```
score = model.evaluate(x_test, y_test, verbose=0)  
print('Test loss:', score[0])  
print('Test accuracy:', score[1])
```

8. Save the model (the file might be ended with extension of .h5 or .weight.h5 for different keras version).

```
model.save_weights('./F-MNISTFilter')# or model.save_weights('./F-MNISTFilter.weight.h5')
```

2. **Application 2** is to use the saved model to visualise the trained filter and respective filters. The analysis will be done on these information following these steps:

1. Call inspect function on the main function.

```
inspect()
```

2. Load the datasets.

```
x_train,y_train, x_test, y_test,input_shape, num_classes = process_data()
```

3. Define the model and load the weights that we saved during training step.

```
model = CNN(input_shape,num_classes)  
model.load_weights('./F-MNISTFilter')# or model.load_weights('./F-MNISTFilter.weight.h5')
```

4. Perform filter explorations by iterating the each of model layer and print the respective layer name. Also, print the weights and biases of convolutional layers inside.

```
for layer in model.layers:  
    print(layer.name)  
    if 'conv' in layer.name:  
        filters, biases = layer.get_weights()  
        print(filters.shape, biases.shape)
```

Question 1: Why some layers have weight and biases, while some do not?

5. Directly access the first layer using indexing of model.layers properties (use variable **indexing** to hold the indexer). Print the shape of accessed weights and biases and its contents.

```
indexing = 0
filters, biases = model.layers[indexing].get_weights()
print(f"Filter Shape : {filters.shape}")
print(f"Filters : {filters}")
print(f"Bias Shape : {biases.shape}")
print(f"Bias : {biases}")
```

6. Normalise the filter between 0 to 1 for visualisation and print the filters before and after normalisation.

```
f_min, f_max = filters.min(), filters.max()
print(f'Before Normalisation, Min = {f_min} and Max = {f_max}')
filters = (filters - f_min) / (f_max - f_min)
print(f'After Normalisation, Min = {filters.min()} and Max = {filters.max()}')
```

7. Plot the selected set of filter using plot_filter() function, and save them for future references. In this case, the plotting uses 4 column with number of row depends on the total number of filter divided by number of column:

```
plot_filter(filters, 4, int(truediv(filters.shape[-1],4)))
```

Question 2: What can you say about these filters? what the features extracted with this filter will look like?

8. Get the layers to a list for successive features extractions. Use indexing on the model.layers to access the concerned layers and print them for verifications. In this case, we include all operations from first convolution (model.layers[0]) to final fully connected layer (model.layers[5]):

```
layer_outputs = [layer.output for layer in model.layers[0:7]]
print(f"Layer Outputs: {layer_outputs}")
```

9. Use the defined output layers as parameter to current model, and use model.input as input parameters. This will create a bridge function (activation_function variable in this example) that allows us to inspect intermediate layer during successive operations defined beforehand.

```
activation_model = Model(inputs=model.input, outputs=layer_outputs)
```

10. Get an example of test data (e.g. the 30th data from test sample) from x_test (through indexing) and put them to a variable.

```
example = 30
img_tensor = x_test[example]
```

11. Plot the example image using plotImage() function (imported from utils.py).

```
plotImage(img_tensor)
```

12. Expand a first dimension to enable batching.

```
img_tensor = np.expand_dims(img_tensor, axis=0)
```

13. Perform predicition using the defined activation_model (through predict() function).

```
activations = activation_model.predict(img_tensor)
```

14. Inspect the resulting activations by first printing the number of layer of activations.

```
print("Number of layer activations: " + str(len(activations)))
```

Question 3: why do you have these number of layers? Which one goes to which (recall your layer_outputs definition on application Application 2 step 8 and compare them with the results from Application 2 step 4)?

15. Get first activations through array indexer by using indexing variable of 0, and print the shape.

```
sel_activation = activations[indexing]
print(f"First Layer Activation : {sel_activation.shape}")
```

Question 4: why does the activation of this layer have this size?

16. Compare your activations shape with the sub output from model.summary() for clarifications.

```
print(model.summary())
```

17. Plot a single feature (the last features of index 32) from this first activation layer.

```
plt.matshow(sel_activation[0, :, :, -1], cmap='summer')
plt.legend()
```

Question 5: Why do the features have these appearances? is there any correlation with the respective filter? Compare with your saved filter figure that you retrieved earlier on step 7.

18. Use display_activation function to plot all 32 features, as results from convolution of learned 32 filters from the first convolutional layer.

```
display_activation(sel_activation, 4, int(truediv(sel_activation.shape[-1],4)))
```

Question 6: By comparing with the filters that you saved earlier, can you corroborate on why the features are appear on that way?

Exercise 1: Change the examples (defined on Application 2 step 10) and observe the extracted features on step 18 (do at least for 3 different examples) for variety of these samples.

Exercise 2: Repeat step 5 to step 18 by changing the selected convolutions to second convolutions (by changing the variable **indexing** at step 5). This will result in different filters extracted at step 7 and associated features on the step 18. Then comment on the differences on the extracted features between the one extracted now (second convolution) to the first convolution. Are there any changes and differences? Do for at least a sample of input.

3. **Application 3** GradCam applications.

1. Application 3 Step 1, call function gradCam()

```
gradCam()
```

2. Application 3 step 2, load the dataset, model and its weights (Application 2 Step 2 and step 3)

```
x_train,y_train, x_test, y_test,input_shape, num_classes = process_data()
```

```
model = CNN(input_shape,num_classes)
model.load_weights('./F-MNISTFilter')# or model.save_weights('./F-MNISTFilter.weight.h5')
```

3. Application 3 Step 3, target the last layer to attach the gradient calculation hook.

```
layer_name = 'conv2d_2'
```

4. Application 3 step 4 try to test some image to calculate the associated heatmap.

```
for i in range(5):
    experiment = x_test[[i+10*i]]
    make_heatmap = calc_gradcam_heatmap(experiment, model, layer_name)

    plt.imshow(superimpose_image(experiment, make_heatmap))
    plt.show()
```

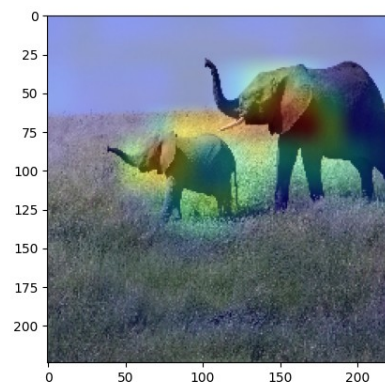
Question 7: Why the models focus differ for each image? And why does it focus on the specific areas?

4. **Application 4**, repeat the Application 1 – 3 using the FashionMNIST dataset. This can be done by changing the dataset used on process_data

```
(x_train, y_train), (x_test, y_test) = fashion_mnist.load_data()
```

Do not forget to change the name of saved models, and also answer each question and exercise on this data.

5. **Application 5**, Run the **GradcamTL.py** and try two different images (with different targeted labels) to evaluate the model focus.



OBSERVATION: The laboratory report (*.pdf file) will contain:

- The code for all the exercises. The code lines introduced in order to solve an exercise can be marked with a different color and can be explained with comments.
- Print screens with the results displayed in the PyCharm/Editor console.
- The exercises solutions: tables of results and responses to the questions if any.